SUB-TERRANEAN NAVIGATION PROJECT

INTRODUCTION

Welcome to the Sensor Fusion Course Final Project! In this exciting challenge, your team will step into the role of underground exploration, racing through mysterious tunnels without GPS. Your mission? Build a state estimation algorithm that keeps track of your robot's position and orientation using onboard sensors and Time-of-Flight (ToF) distance measurements, just like navigating an uncharted sub-terranean world.

OVERVIEW AND OBJECTIVES

- **Team Size:** 4 students per team
- **Core Task:** Estimate the robot's 2D position (x, y) and orientation (yaw angle) in real time, inside a motion capture arena.
- **Primary Hardware:**
  - **STM Nucleo IKS02A1 board** featuring:
    - ISM330DHCX (3-axis accelerometer + 3-axis gyroscope)
    - IIS2MDC (3-axis magnetometer)
    - IIS2DLPC (3-axis low-power accelerometer)
  - **X-NUCLEO-53L1A1** with three ToF sensors (one forward-facing, two sideways-facing).
- **Mobile Base carrying the Nucleo board**
  - **The platform can move sideways or rotate with Mecanum wheels**

Figure 1 Mobile Base carrying the Nucleo board:

ENVIRONMENT SETUP

- **Arena**: The robot operates inside a motion capture facility with walls. The arena's dimensions and the robot's starting point will be provided beforehand.
- **Walls with Holes**: Some walls contain holes, potentially causing erroneous ToF readings (simulating real-world sensor noise/failures).
- **Ground Truth**: A PhaseSpace camera system with reflective markers on the robot will track the "true" position and orientation. This will be used to assess the accuracy of your algorithm.

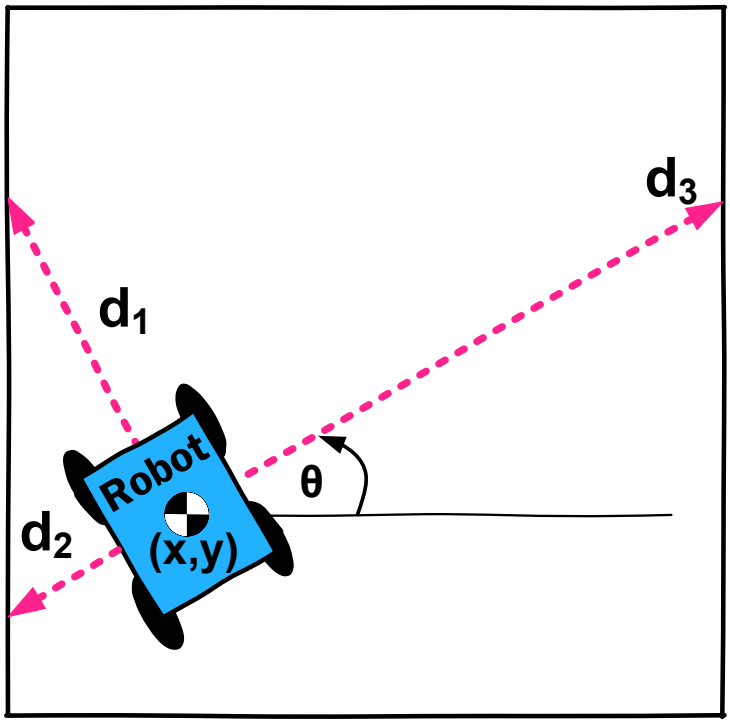

Figure 2: The robot will be equipped with three ToF sensors measuring the distances to the walls. The robot will also have IMU sensors.

To help you design your Extended Kalman Filter (EKF) and write your final report, we will provide collected datasets for you to run your estimator on during the development phase.

1. **Training data 1:** We will provide a .mat file with logged data of the robot doing task 1's motion. This includes all sensor data (IMU, magnetometer, temperature, and 3x ToF) as well as the ground truth PhaseSpace data.
2. **Training data 2:** We will provide a .mat file with logged data of the robot doing task 2's motion.
3. **Calibration data 1:** A .mat file with logged data of the robot standing stationary for 1 minute, driving completely straight forwards and backwards.
4. **Calibration data 2:** A .mat file with logged data of the robot continuously rotating for 5 complete cycles.

Note: The datasets have ground truth position and orientation, which is useful for evaluating your estimation result. You will need to use the first few points for the starting position. Do not assume the robot starts at (0,0,0). The origin is defined as the centre of the box.

The ground truth rotation is in quaternions (4 numbers). More detail can be found at: https://uk.mathworks.com/help/nav/ref/quaternion.html

Dataset Structure Example

Struct Name	Dimension	Update Frequency	Elements in Value
GT_position	[3]	200 Hz	X, Y, Z [m]
GT_rotation	[4]	200 Hz	W, X, Y, Z (Quaternion)
GT_time	[1]	200 Hz	Time [s]
Sensor_ACCEL	[3]	104 Hz	ISM330DHCX Acceleration X, Y, Z [m/s²]
Sensor_GYRO	[3]	104 Hz	ISM330DHCX Gyroscope X, Y, Z [rad/s]
Sensor_LP_ACCEL	[3]	100 Hz	IIS2DLPC Acceleration X, Y, Z [m/s²]
Sensor_MAG	[3]	50 Hz	IIS2MDC Magnetometer X, Y, Z [T]
Sensor_Temp	[1]	100 Hz	Temperature [°C]
Sensor_Time	[1]	200 Hz	Time [s]
Sensor_ToF1	[4]	10 Hz	Distance[m], Ambient[kcps/spad], Signal[kcps/spad], Status
Sensor_ToF2	[4]	10 Hz	Distance[m], Ambient[kcps/spad], Signal[kcps/spad], Status
Sensor_ToF3	[4]	10 Hz	Distance[m], Ambient[kcps/spad], Signal[kcps/spad], Status

(Note: Please refer to the sensor's datasheet and user manual to determine the definitions of the ToF Range status codes.)

PROJECT TASKS

1. **Straight-Line Drive:** The robot moves forward and backward in a straight line at known intervals. Your algorithm must accurately estimate position and yaw during these simple movements.

2. **Circuit Course Drive:** The robot drives a more complex path around the arena with varying velocity. The exact path is unknown to you ahead of time and differs across teams. Your estimator must remain robust to changing speed and direction.

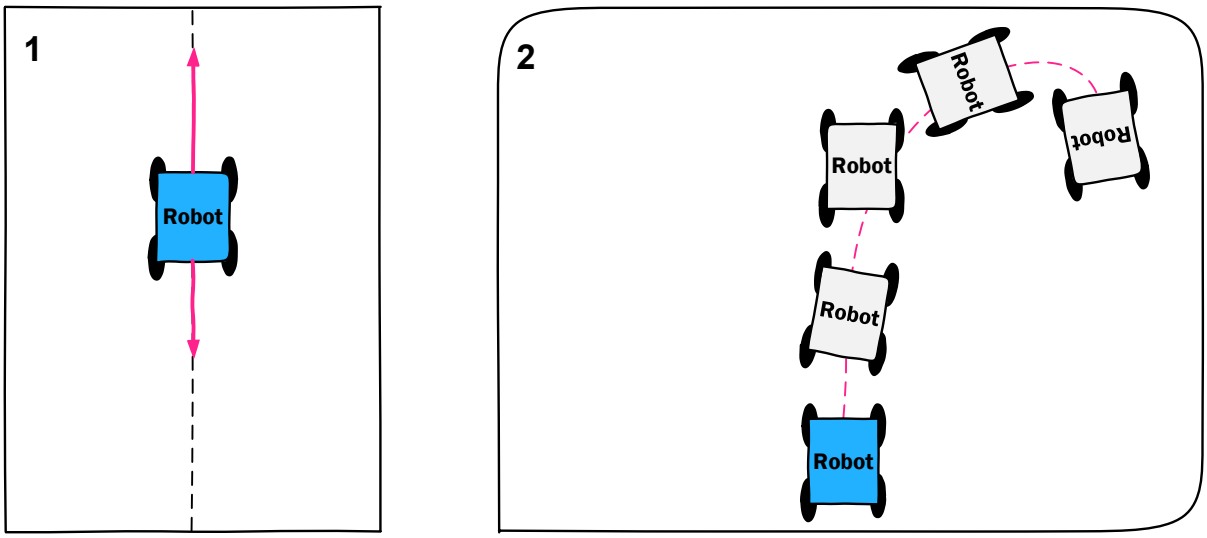

Figure 3: The estimation tasks for the project that need to be completed by your team.

DELIVERABLES

- **Code:** Your finalized code for the state estimation and control interface.
  - **Requirement of the code**
    - You should submit **ONE SINGLE function file** named **myEKF.m**
    - The function must have the following format:

```
%Your function should take in the logged struct and
output your state estimate as well your state covariance
and the ground truth (X,Y,Z).
[X_Est, P_Est, GT] = myEKF(out)
```

    - Please do not:
      - Rename the function (it must be called myEKF)
      - Change the file name (it must be myEKF.m)
      - Call external files or scripts from within the function
      - Include file paths in your code
    - We will only run the myEKF function inside myEKF.m to get the X_Est and P_Est
    - You may define additional helper functions inside the same myEKF.m file and call them from within myEKF.
    - The same code must be used for both Task 1 and Task 2.
- You will need to run your estimation using the data and submit a .mat file with your state vector which we will use for the grading.
- **6-Page Report:** Explaining the design of your state estimator, sensor fusion methodology, any relevant algorithms and your results on the training data.

    - A detailed template will be released in the coming weeks.
    - Make sure to detail how you designed your estimator to tackle sensor noise, calibration, distance anomalies, real-time constraints, etc.
- **Submission**: The report and code should be submitted on to Moodle as a .zip file
    - **File check list**:
        - 1 x PDF of the report
        - 1 x myEKF.m **FUNCTION** file
        - All of your .mat files for your estimation – Please name your .mat files with the name of the task, e.g. task1_1_estimated.mat

DELIVERABLES

- **Practical Performance (70%)**
    - Performance in Task 1, Task 2 (each weighted proportionally to difficulty) at the live demo. This will be determined by comparison to motion capture ground truth.
    - Code quality (10% of practical mark)
        - This component evaluates how easy it is for us to run and understand your code. We will consider:
            - Ease of execution – Your code should run without requiring us to modify it
            - Portability – Avoid hard-coded file paths or machine-specific settings that will not work on other computers
            - Clarity and readability – Code should be clearly structured and easy to follow
            - Documentation – Include comments that explain the purpose of key parts of the code
    - Task performance (90% of the practical mark)
        - Task performance is assessed using the absolute RMSE from the X_Est and P_Est generated by your code, compared with the ground truth
        - Task 1: 40% of practical mark
        - Task 2: 50% of practical mark
    - For each task:
            - Submissions will be ranked according to their absolute RMSE.
            - The highest RMSE will receive 80% of the marks for that task
            - The lowest RMSE will receive 50% of the marks for that task
            - Marks for other submissions will be scaled proportionally based on their ranking
- **Report (30%)**
    - Clarity and depth of your explanation.
    - Quality of experimental results and analysis.
    - Insights into design trade-offs and lessons learned.